CSCI 6461 | TOPIC 10

Memory Hierarchy, Locality, & Cache Fundamentals

AArch64 memory accesses, cache organization, write policies, and performance

Dr. J. Burns

Topic 10 Outline

1. **Memory Hierarchy and Locality**
   - Cache lines, AArch64 access widths, temporal/spatial locality, and stride.
2. **Cache Organization and Address Mapping**
   - Tags, sets, ways, lookup, replacement, and the three classes of cache miss.
3. **Stores and Cache Performance**
   - Line fills, write policies, dirty eviction, and average memory access time.
4. **AArch64 Program Behaviour**
   - Traversal order, dependent accesses, memory attributes, and prefetch hints.

PART 1

Memory Hierarchy and Locality

Why Use a Memory Hierarchy?

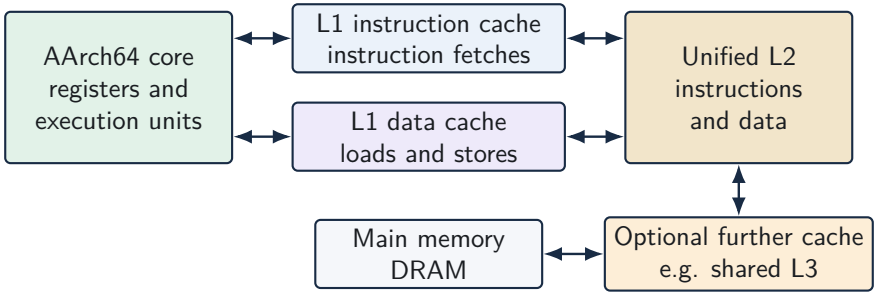
- A core needs instructions and data quickly, but large memories are costly to access in both time and energy.
- Small, fast storage near the core holds the values most likely to be needed soon.
- Larger, slower levels provide capacity. Caches reduce the **average** cost of accessing that capacity.
- AArch64 `ldr` and `str` specify memory effects; their latency depends on where the data is found.

Two distinct performance measures

Latency: time to complete an access. **Bandwidth**: data transferred per unit of time.

All numerical cache configurations in this topic are illustrative. One KiB is 1024 bytes.

From AArch64 Registers to Main Memory

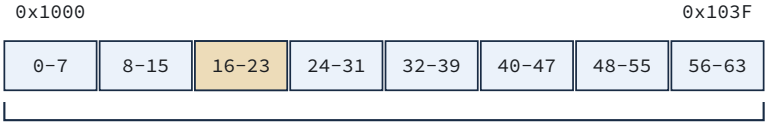


- Typical organization: separate L1 instruction/data caches; unified lower levels. Sizes, sharing, and levels vary by implementation.
- Caches commonly use SRAM (static RAM); main memory uses denser DRAM (dynamic RAM).
- SSD/disk storage is accessed through software and I/O. A normal cache miss does not directly fetch a line from disk.

A Cache Line Contains Neighbouring Bytes

Example assumptions: 64-byte lines; cacheable Normal memory; valid aligned load; `x0=0x1000`; line initially absent.

`ldr x2, [x0, #16]` requests bytes `0x1010`–`0x1017`.



one aligned 64-byte line; cell labels are byte offsets

- **Access width: 8 bytes. Line-fill size: 64 bytes.** The cache fetches the line at `0x1000`.
- A 64-byte line holds eight 64-bit values, sixteen 32-bit values, or sixteen 4-byte A64 instructions.
- Neighbouring accesses can reuse the fetched line while it remains resident.
- Line base $= \lfloor \text{address}/64 \rfloor \times 64$. A line boundary is independent of source-level object boundaries.

Temporal and Spatial Locality

Example assumptions: `x0=0x1000`; `x1=8`; 64-byte data-cache lines; valid aligned loads from cacheable Normal memory.

One AArch64 loop

```
.Lscan:
ldr x2, [x0], #8
subs x1, x1, #1
b.ne .Lscan
```

Each `ldr` reads eight bytes, then advances the data pointer `x0` by eight bytes.

Instruction temporal locality

Each iteration reuses the same `ldr`, `subs`, and `b.ne` instructions. Their **instruction addresses repeat**.

Data spatial locality

The loads access `0x1000`, `0x1008`, ..., `0x1038`. These **data addresses advance** through eight adjacent values in one 64-byte line.

Two reference streams from one program: instruction fetches demonstrate reuse over time; data loads demonstrate access to neighbouring locations.

Either form of locality can occur in instruction or data accesses; this example explicitly identifies one of each.

Sequential AArch64 Loads Reuse One Line

Example assumptions: Cold cache; 64-byte lines; aligned base `0x1000`; eight 64-bit values; no prefetching.

Pointer advances by eight bytes

```
.Lsum:
ldr x2, [x0], #8
add x3, x3, x2
subs x1, x1, #1
b.ne .Lsum
```

Initially `x1=8`, `x3=0`. Post-indexing uses the old `x0`, then adds 8.

Load address	Data-cache result
`0x1000`	miss; fetch the line
`0x1008`–`0x1038` in steps of 8	seven hits in that line

Eight data loads: **one miss, seven hits**. Miss rate $= 1/8 = 12.5\%$.

The loop's instruction fetches are separate cache references. The count above describes only its eight data loads.

Stride Determines How Much of Each Line Is Used

Stride is the difference, in bytes, between successive memory-access addresses.

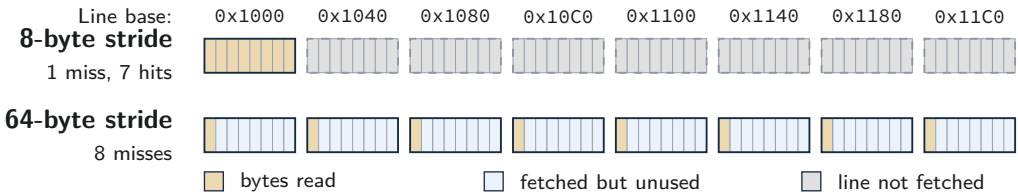
Example assumptions: Each row is a separate eight-iteration loop; 8-byte loads; 64-byte cache lines; cold cache; base `0x1000`; no prefetching.

Address stride	Load repeated eight times	Lines fetched	Useful bytes / fetched bytes
8 bytes	`ldr x2, [x0], #8`	1	$64/64 = 100\%$
64 bytes	`ldr x2, [x0], #64`	8	$64/512 = 12.5\%$

- In each loop, the shown instruction executes **eight times**, reading **8 bytes per iteration**. Its post-index immediate advances `x0` by 8 or 64 bytes after each load.
- **Impact here:** stride 8 uses all eight values in one fetched line. Stride 64 uses one value per line: eight times the fill traffic for the same 64 useful bytes.
- The **working set** is the collection of lines needed over a period of execution.

Visualizing Stride: Line Use and Performance

Example assumptions: Same two loops; cold cache; no prefetching. Each 64-byte line has eight 8-byte cells.



- **Performance:** more misses can expose memory latency; more line-fill bytes consume bandwidth. Eight times the traffic does not imply eight times the execution time.
- **Cache pressure:** stride 64 needs eight lines for 64 useful bytes; stride 8 needs one. On repeated scans, limited capacity can evict lines before reuse.
- **Thrashing**: repeatedly evicting and refetching needed lines. The eight first-pass misses above do not by themselves demonstrate thrashing.

Capacity, Line Size, Sets, and Ways

Cache data capacity

$$C = S \times A \times B \qquad \Longrightarrow \qquad S = \frac{C}{AB}$$

Parameter	Meaning	Main worked configuration
C: capacity	data bytes stored; metadata excluded	32 KiB $= 32{,}768$ bytes
B: line size	bytes in one line	64 bytes
A: associativity	lines (ways) in each set	4 ways
S: set count	independently indexed sets	$32{,}768/(4 \times 64) = 128$

This cache holds $128 \times 4 = 512$ lines. Each memory line has one candidate set and can occupy any of its four ways.

Mapping example: 48-bit physical addresses; assume identity translation and ordinary low-bit indexing. No prefetching or other-core traffic. Smaller caches are labelled separately.

Split the Address into Tag, Set Index, and Offset

Example assumptions: Byte address a; 48-bit physical addresses; 32 KiB data; 4 ways; 64-byte lines; 128 sets.



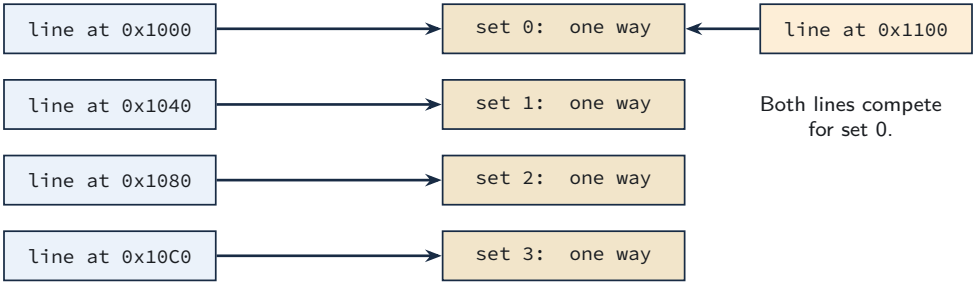
$$\text{line number} = \left\lfloor \frac{a}{64} \right\rfloor, \quad \text{set} = \text{line number} \bmod 128, \quad \text{tag} = \left\lfloor \frac{\text{line number}}{128} \right\rfloor$$

- The offset selects a byte within a line; the index selects a set; the tag identifies which memory line occupies a way.
- Address `0x1238`: line number 72, set 72 (`0x48`), tag 0, byte offset 56 (`0x38`).
- `ldr x2, [x0]` with `x0=0x1238` reads the last eight bytes of the line at `0x1200`.

The 48-bit width is an example physical-address width. An AArch64 X address register remains 64 bits wide.

Direct Mapping: One Possible Cache Location

Example assumptions: Small example: 256-byte data cache; four 64-byte lines; one way per set.



- Set index $= \lfloor a/64 \rfloor \bmod 4$. Addresses separated by 256 bytes have the same index.
- A valid line with a matching tag gives a **hit**; otherwise the access is a **miss**.
- Set 0 cannot hold the lines at `0x1000` and `0x1100` simultaneously, even when other sets are empty.

A Direct-Mapped Trace: Hit, Then Conflict

Example assumptions: 256-byte direct-mapped cache; 64-byte lines; initially all valid bits are 0.

Each address below is used by `ldr x2, [x0]`.

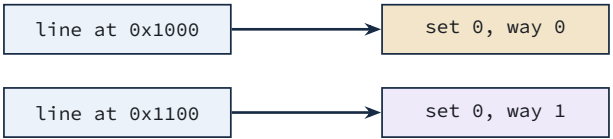
Address	Set	Tag	Result	Set 0 after access
`0x1000`	0	16	miss	valid; line at `0x1000`
`0x1008`	0	16	hit	same line; different offset
`0x1100`	0	17	miss	replace with line at `0x1100`
`0x1000`	0	16	miss	replace with line at `0x1000`

- First access to each distinct line is compulsory. The final miss is a conflict miss: two lines fit in total capacity but collide in one set.
- Hit rate $= 1/4 = 25\%$; miss rate $= 3/4 = 75\%$.
- Compare tags only when the valid bit is set. An uninitialized tag value cannot create a hit.

Associativity Gives a Line More Candidate Locations

Example assumptions: Hold capacity at 256 bytes and line size at 64 bytes.

Organization	Sets	Ways per set	Candidate locations
Direct mapped	4	1	one line in one selected set
2-way set associative	2	2	either way in one selected set
Fully associative	1	4	any of the four lines

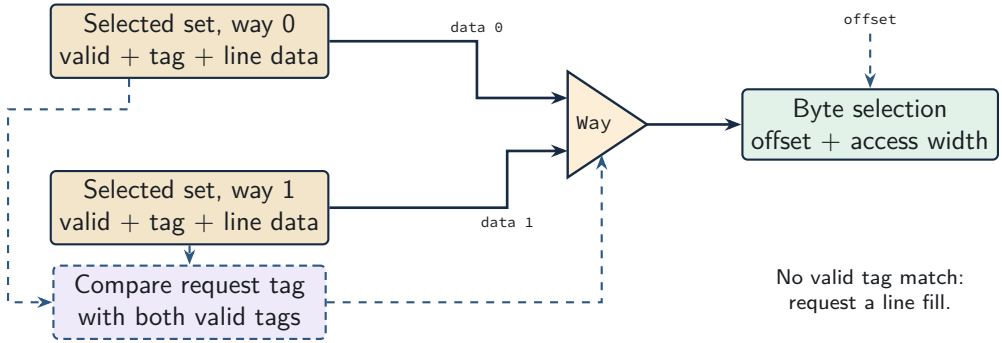


2-way example: both lines can stay in the same set.

More ways reduce placement restrictions, but require more tag comparisons and data selection hardware.

Cache Lookup: Select a Set, Compare Tags, Select Data

Example assumptions: Conceptual 2-way lookup; the offset selects bytes within the matching line.



- `Hit = (V0 & tag0==requestTag) | (V1 & tag1==requestTag)`.
- The triangle selects the matching way's data; tags distinguish lines sharing a set.

Replacement: Choose an Invalid Way, Then a Victim

Example assumptions: 256-byte cache; two sets; two ways; 64-byte lines; LRU replacement; initially empty.

Victim: a resident cache line selected for eviction.
LRU: least recently used. A hit makes its line most recently used.

Load address	Result	Set 0 contents	Action
`0x1000`	miss	`0x1000`, empty	use an invalid way
`0x1080`	miss	`0x1000`, `0x1080`	use the other invalid way
`0x1000`	hit	`0x1000`, `0x1080`	`0x1080` is now LRU
`0x1100`	miss	`0x1000`, `0x1100`	evict `0x1080`
`0x1080`	miss	`0x1080`, `0x1100`	evict `0x1000`

- All three lines compete for two ways in set 0; set 1 remains unused.
- Placement chooses the set; replacement chooses the line that leaves.
- These traces use LRU. AArch64 does not mandate a replacement algorithm.

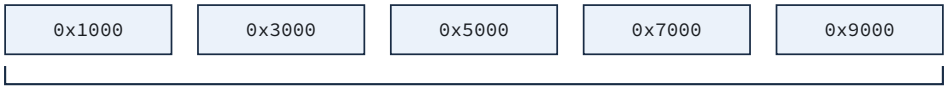
Why Did the Access Miss? The Three Cs

Miss class	Cause	Possible improvement
Compulsory	First reference to a line in an initially cold cache.	Use more bytes from each fetched line; prefetch may hide its latency.
Capacity	Reuse fails even in a same-capacity fully associative LRU cache.	Reduce the active working set or increase capacity.
Conflict	Reuse fails here, but would hit in that fully associative LRU cache.	Increase associativity or change the address layout.

- Classify first references as compulsory before considering capacity or conflict.
- The trace and cache organization together determine the miss rate.
- Coherence invalidations can cause additional misses; they are outside these traces.

A Small Working Set Can Still Thrash One Set

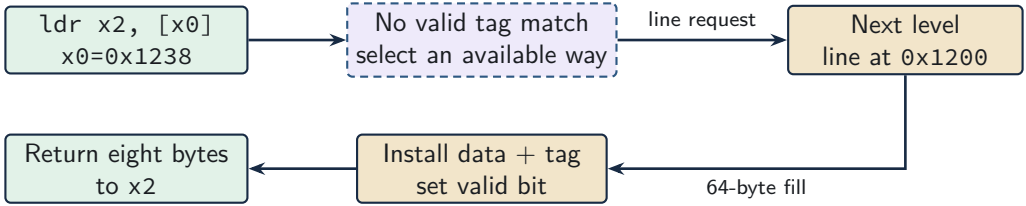
Example assumptions: Main cache: 32 KiB, 4 ways, 64-byte lines, 128 sets, LRU. Repeat the five loads below.



- Address spacing is `0x2000` $= 8192 = 128 \times 64$ bytes: identical set index, different tags.
- Only $5 \times 64 = 320$ data bytes are needed, but that set has four ways. Every access misses during the repeated round-robin trace.
- After the compulsory misses, these are **conflict misses**: a 512-line fully associative cache would retain all five lines.
- By contrast, repeatedly scanning 40 KiB of distinct lines exceeds the 32 KiB capacity; a fully associative LRU cache also misses on the next scan.

A Read Miss Fetches a Line from the Next Level

Example assumptions: Blocking cache; read allocation; 64-byte lines; valid memory translation; no prefetching.



- The next level can be L2, another cache, or DRAM; an L1 miss need not reach DRAM.
- Before evicting a line with pending writes (a **dirty victim**), write it back. Clean or invalid lines need no data writeback.
- Return bytes `0x1238`–`0x123F` from the fetched line at `0x1200`.
- A cache miss changes timing; it does not itself imply a page fault.

Store Hits: Write-Through or Write-Back

Example assumptions: `str x2, [x0, #8]`; `x0=0x1000`; the 64-byte line is already present.



Only the addressed eight bytes change; the other bytes are preserved.

Hit policy	Action	Consequence
Write-through	Update the cached bytes and send the write to the next level.	More outgoing write traffic; buffering can decouple the core from it.
Write-back	Update the cached bytes and set a dirty bit.	The newest bytes may exist only in this cache until writeback.

A **dirty** line may differ from the next level's copy. Eviction requires writeback; maintenance or coherence may also cause writeback in real systems.

Store Misses: Allocate a Line or Bypass This Cache

`str x2, [x0]` targets a line that is absent.

Write allocate

1. Obtain the line from the next level.
2. Install it in this cache.
3. Update the store's bytes.

With write-back, mark the installed line dirty. A later access can hit while it remains resident.

No write allocate

1. Send the write to the next level.
2. Do not install the missed line in this cache.

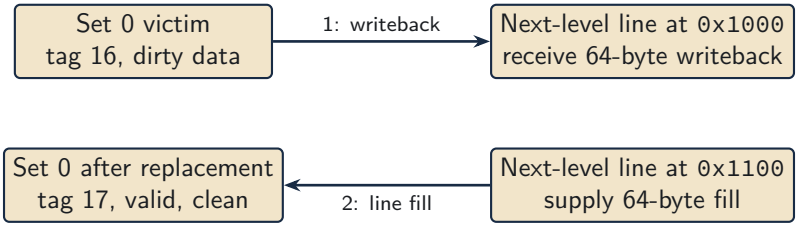
A later load must obtain the line if it is still absent. The next level follows its own policy.

- Write-through/write-back describes **write hits**; allocation describes **write misses**. They are separate choices.
- Worked store traces use **write-back + write allocate**, with full-line fills and writebacks. Optimized full-line stores are outside this model.

Dirty Eviction Uses the Victim's Address

Example assumptions: 256-byte direct-mapped cache; 64-byte lines; write-back. Set 0 holds dirty line `0x1000`.

A load at `0x1100` needs the same set. The old line's tag is 16; the new tag is 17.



- Reconstruct the victim line base from **its saved tag and set index**: $(16 \times 4 + 0) \times 64 = \texttt{0x1000}$.
- Writing dirty data to the new request's address would corrupt a different memory line.
- At this cache boundary: clean read miss transfers 64 fill bytes; dirty read miss transfers $64 + 64 = 128$ data bytes. Control traffic is excluded.

Average Memory Access Time: Hits and Misses

Average memory access time (**AMAT**) combines the hit cost with the additional miss cost.

One cache level

$$\mathrm{AMAT} = t_{\mathrm{hit}} + m \times p_{\mathrm{miss}}$$

m: misses / accesses. p_{miss}: **additional** time after the hit lookup fails.

One nanosecond (ns) is 10^{-9} seconds. Timings below are illustrative.

Configuration	Average
Hit time 1 ns; miss rate 5%; penalty 60 ns	$1 + 0.05 \times 60 = 4$ ns
Same timings; miss rate reduced to 1%	$1 + 0.01 \times 60 = 1.6$ ns

- A small fraction of expensive misses can dominate the average.
- This serial model excludes overlap and queuing. AMAT is not time per instruction.

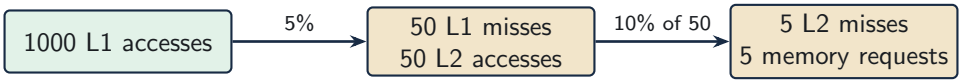
Two Levels: Local Miss Rate and Global Miss Rate

Example assumptions: L1 hit time 1 ns; L1 miss rate 5%; L2 lookup 8 ns; L2 local miss rate 10%; additional memory cost 60 ns.

Only L1 misses consult L2

$$\text{AMAT} = t_1 + m_1(t_2 + m_2 p) = 1 + 0.05(8 + 0.10 \times 60) = 1.7 \text{ ns}$$

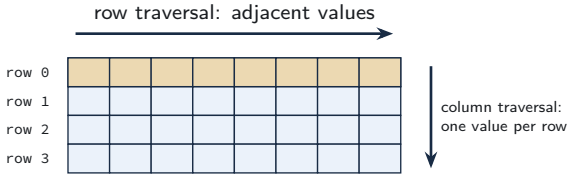
t_1, t_2: lookup times; m_1, m_2: local miss rates; p: additional memory cost.



- L2 **local** miss rate $= 5/50 = 10\%$: denominator is L2 accesses.
- L2 **global** miss rate $= 5/1000 = 0.5\% = m_1 m_2$: denominator is original accesses.
- Average extra cost after an L1 miss is $8 + 0.10 \times 60 = 14$ ns. Do not apply that cost to every L1 hit.

Traversal Order Changes Line Reuse

Example assumptions: Row-major 4×8 array of 64-bit values; base `0x4000`; 64-byte rows; cold fully associative 2-line LRU cache; no prefetching.



One row = one line.
Only two rows fit.

Row inner loop: 8-byte stride

```
ldr x2, [x0]
add x0, x0, #8
```

All 32 loads: **4 misses, 28 hits**.

Column inner loop: 64-byte stride

```
ldr x2, [x0]
add x0, x0, #64
```

All 32 loads: **32 misses, 0 hits**.

Outer loops choose row/column starts. If all four lines stay resident, both traversals have four compulsory misses.

Streaming Loads and Dependent Pointer Loads

Contiguous values

```
ldr x2, [x0]
ldr x3, [x0, #8]
ldr x4, [x0, #16]
```

- Addresses are available from the same base.
- Nearby values can share a cache line.
- A capable core can overlap independent requests; the ISA does not guarantee this.

Linked nodes; next pointer at offset 0

```
ldr x0, [x0]
ldr x0, [x0]
ldr x0, [x0]
```

- Each load supplies the next load's address.
- Scattered nodes may have little spatial locality.
- The dependency limits overlap along this chain; resident nodes can still hit.

High bandwidth helps streams of independent transfers. A dependent chain is especially sensitive to access latency.

AArch64 Cache Behaviour and Prefetch Hints

Optional software prefetch

```
prfm pldl1keep, [x0, #64]
...
ldr x2, [x0, #64]
```

- `PLDL1KEEP`: prefetch for a future load, targeting L1 with a temporal retention hint.
- `prfm` has an implementation-defined effect. It neither loads `x2` nor guarantees residency; the `ldr` is still required.
- Memory attributes make Normal memory cacheable or non-cacheable. Device memory is non-cacheable.
- Write policies depend on memory attributes and the processor implementation; `ldr`/`str` do not select them.

Cache sizes and line lengths vary by implementation; 64-byte lines are examples. Our single-core traces omit translation, coherence, and device ordering.

Topic 10 Summary

- **Hierarchy:** smaller, faster levels reduce average access cost; larger levels provide capacity.
- **Locality:** revisit resident lines and use neighbouring bytes. Access width, line size, stride, and working set are different quantities.
- **Organization:** capacity = sets $\times$ ways $\times$ line bytes. Index selects a set; valid tag matches identify a hit.
- **Misses:** distinguish compulsory, capacity, and conflict. More capacity alone does not remove every placement problem.
- **Stores:** write-hit and write-miss policies are separate; dirty victims must preserve their bytes at the correct next-level address.
- **Performance:** use consistent AMAT costs and miss-rate denominators. Locality and dependency shape actual program behaviour.

References: Arm-authored A64 instruction specifications; Arm memory systems; Cornell cache organization; Berkeley AMAT.

Self-Test 1: Locality and Address Mapping

1. Explain why `ldr x2, [x0]` reads eight bytes even when a miss fills a 64-byte cache line. How many 64-bit values fit in that line?

2. From aligned base `0x2000`, eight loads use an 8-byte stride. Give the addresses, line count, and cold-cache miss rate. Repeat with a 64-byte stride.

3. A 32 KiB cache has 64-byte lines and four ways. Calculate its line count, set count, and tag/index/offset widths for a 48-bit physical address.

4. For address `0x3238` in that cache, calculate the line base, offset, set index, and tag. Why does it share a set with `0x1238`?

5. Which locality does repeated instruction fetch in a loop exploit? Why does repeatedly adding to an X-register accumulator create no data-cache references?

Self-Test 2: Mapping, Replacement, and Stores

Example assumptions: 64-byte lines; each trace starts with an empty cache.

1. Trace loads at `0x1000`, `0x1008`, `0x1100`, `0x1000` in a 256-byte direct-mapped cache. Give hit/miss outcomes and miss classes.
2. Repeat with a 256-byte 2-way cache. Explain which outcome changes and why the total data capacity is unchanged.
3. In that 2-way LRU cache, trace `0x1000`, `0x1080`, `0x1000`, `0x1100`, `0x1080`. Identify each victim.
4. A store misses in a write-back, write-allocate cache. State the line-fill, byte-update, and dirty-bit actions. Contrast no write allocation.
5. A dirty line at `0x1000` is replaced by line `0x1100`. Give the writeback address and data-byte traffic at this boundary, assuming a full 64-byte writeback and fill.

Self-Test 3: Performance and Program Behaviour

1. Hit time is 2 ns, miss rate 4%, and additional miss penalty 50 ns. Calculate AMAT. What would it be if the miss rate were 1%?

2. L1 hit time is 1 ns and miss rate 8%; L2 lookup takes 10 ns and its local miss rate is 25%; memory adds 80 ns. Calculate AMAT and the global L2 miss rate.

3. Traverse the row-major 4×8 array under the two-line LRU example. Explain the row/column miss counts. What changes if all four rows remain cached?

4. Why can `ldr x0, [x0]` repeated along scattered linked nodes be latency limited even when the memory system supports high bandwidth?

5. Does `prfm pldl1keep, [x0, #64]` guarantee that a later load hits? Explain why the same cache assumptions cannot be applied to Device memory.